

Recursion

Christopher Martin - *Bowdoin College* 
c.martin@bowdoin.edu 

Down the Rabbit Hole

Recursion

A problem-solving technique defined by breaking a problem down in terms of a simpler version of itself - viewed as a **divide and conquer** approach

This idea of making a problem self-referential can be a little mind-bending at times, we are going to work through this together!

Recursion is most commonly encountered in math as what are called **recurrence relations, let's take a look at one:**

Recursive Code

Recursion happens when a function directly or indirectly calls itself
Wait... calls itself? How would that work?

```
1 def f():  
2     print("Calling f!")  
3     f()  
4  
5 # f()
```

No output received

What would this code output?

How many functions are running?

Cases

In order for recursion to be useful, we need to have at least one **recursive case** and one **base case**

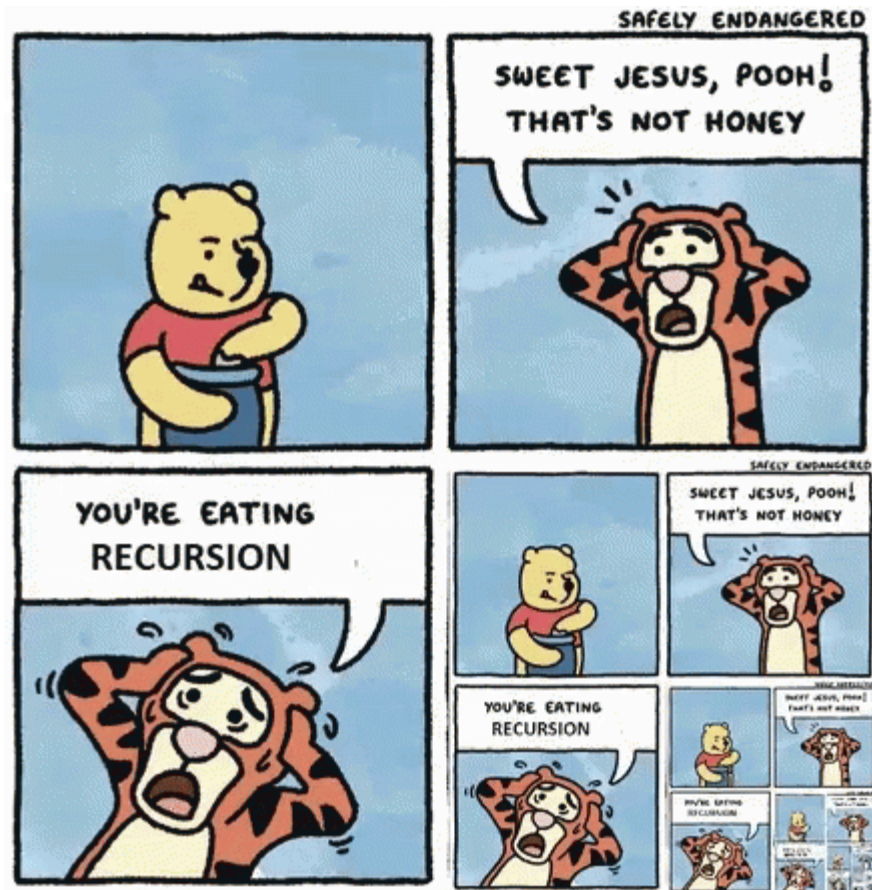
Recursive case

A recursive case is one where we need to recursively break the problem at hand down into a simpler version of itself in order to solve

Without a recursive case, your code just isn't recursive

Base case

A base case is one where the answer to the problem is immediately obvious so no recursion is necessary because no further work is required to solve it



Sum

Let's revisit that `sum` recurrence relation from earlier:

$$sum(n) = \begin{cases} n + sum(n - 1) & \text{if } n > 0. \\ 0 & \text{if } n = 0. \end{cases}$$

What is / are the base case(s)?

What is / are the recursive case(s)?

Is this a proper recursive function?

Iteration vs Recursion

Recursion allows us to repeat code like loops but in a different way

Iteration and recursion are 'equal in power' meaning any problem can be solved using recursion or looping - however, some problems may lend themselves to being solved easier with loops or recursion

As we move forward, I will try to highlight when you would want to use recursion over iteration but this won't become super important until much later in your Computer Science education

Right now, the goal is to just ensure you understand this alternative exists and to showcase some of the situations where it can be helpful!

Sum Implementations

Recursive `sum`

```
1 def sum(n: int) -> int:
2     if n > 0:
3         return n + sum(n - 1)
4     elif n == 0:
5         return 0
6
7
8 # Try with different numbers!
9 print(sum(5))
```

15

Iterative `sum`

```
1 def sum(n: int) -> int:
2     sum = 0
3
4     for i in range(1, n + 1):
5         sum += i
6
7     return sum
8
9 print(sum(5))
```

15

Trace the execution by hand

Can you follow along with both?